

Búsqueda binaria y bisección

Divide y vencerás cuando el espacio está ordenado

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto de 2026

Objetivos de la sesión

Al terminar la clase, usted podrá

- Reconocer la búsqueda binaria y la bisección como algoritmos de divide y vencerás (OA1).
- Plantear la función objetivo de un problema de búsqueda y su formulación recurrente (OA6).
- Implementar por índices la búsqueda binaria y la bisección continua, y justificar su costo $O(\lg n)$ (OA1).
- Transformar un problema de optimización en uno de decisión y resolverlo con búsqueda sobre la respuesta (OA1).

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

El molde de la clase pasada

Divide y vencerás (CLRS, Sección 2.3.1)

Dividir el problema en subproblemas del mismo tipo, conquistar cada uno recursivamente hasta un caso base trivial, y combinar las soluciones parciales. El costo queda descrito por una recurrencia

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n),$$

y la partición es **por índices**: cada llamado dice de dónde a dónde trabaja, sin copiar nada.

Lo que dejó el ordenamiento por mezcla

Bajar partiendo, subir mezclando: $T(n) = 2 T(n/2) + \Theta(n)$, que se resuelve en $\Theta(n \lg n)$. Hoy la recurrencia es más liviana: en cada paso sobrevive *una sola* mitad.

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado**
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

El juego de adivinar el número

Piense un número entre 1 y 100

Yo solo puedo preguntar “¿es mayor que x ?”. ¿Cuántas preguntas necesito, en el peor caso, para acertar?

La cuenta

Cada respuesta descarta la mitad de los candidatos:

$100 \rightarrow 50 \rightarrow 25 \rightarrow 13 \rightarrow 7 \rightarrow 4 \rightarrow 2 \rightarrow 1$. Siete preguntas bastan, porque $2^7 = 128 \geq 100$. Y para un número entre 1 y un millón alcanzan veinte, porque $2^{20} > 10^6$.

La condición escondida

El juego funciona porque “mayor que x ” parte los candidatos en dos bloques ordenados. Esa es la propiedad que hoy vamos a explotar una y otra vez.

El problema

Especificación

Entrada: un arreglo $A[0..N)$ de números ordenado ascendentemente, con $N \geq 1$, y un número v . **Salida:** $\exists p \in [0..N). A[p] = v$.

Ya sabemos resolverlo. . . sin usar el orden

La búsqueda lineal de la clase de invariantes revisa las N posiciones una por una: $O(n)$, y su correctitud quedó certificada. Pero ignora por completo que el arreglo viene ordenado.

Pregunta

¿Cómo se juega aquí el juego de adivinar el número?

La función objetivo

Nombrar lo que se busca

Para $0 \leq l < r \leq N$, se define

$\varphi(l, r)$: determina si v está en $A[l..r)$.

El intervalo es **medio abierto**: incluye l y excluye r , así que su tamaño es $r - l$.

Reformulación de la salida

La salida pedida es exactamente $\varphi(0, N)$: el problema completo es un caso particular de la función objetivo.

La función objetivo

Con $A = [1, 4, 6, 8, 10, 13, 20, 22]$

$\varphi(0, 8)$ pregunta por todo el arreglo; $\varphi(4, 6)$ pregunta si v está en $[10, 13]$, las posiciones 4 y 5.

El planteamiento recurrente

φ se resuelve partiendo el intervalo

Con $mid = \lfloor (l + r)/2 \rfloor$:

$$\varphi(l, r) = \begin{cases} A[l] = v & \text{si } r - l = 1 \\ \varphi(l, mid) & \text{si } r - l > 1 \text{ y } v < A[mid] \\ \varphi(mid, r) & \text{si } r - l > 1 \text{ y } v \geq A[mid] \end{cases}$$

Los tres momentos

Dividir: calcular mid . Conquistar: *un solo* subproblema, la mitad que sobrevive. Combinar: nada, la respuesta del subproblema es la respuesta.
Caso base: el intervalo de un elemento.

Por qué descartar la mitad es seguro

Caso $v < A[mid]$: la mitad derecha no sirve

Todo $p \in [mid..r)$ cumple $A[p] \geq A[mid]$ porque el arreglo está ordenado, y $A[mid] > v$: en la mitad derecha no puede haber un testigo. Los testigos posibles, si existen, están todos en $[l..mid)$.

Caso $v \geq A[mid]$: la mitad derecha basta

Aquí v podría estar en las dos mitades, pero no hace falta revisar la izquierda: si hubiera un testigo $p < mid$, el orden da $v = A[p] \leq A[mid] \leq v$, o sea $A[mid] = v$, y el propio mid —que sí queda en $[mid..r)$ — es testigo. Buscar solo a la derecha no pierde ninguna respuesta.

Por qué descartar la mitad es seguro

Sin orden no hay descarte

Los dos argumentos usan la monotonía del arreglo. Sobre un arreglo sin ordenar, botar una mitad puede botar la única aparición de v .

La implementación

```
def buscar(lista, v, ini, fin):  
    # Determina si v esta en lista[ini..fin), que viene ordenado  
    if fin - ini == 1:  
        resultado = lista[ini] == v  
    else:  
        mitad = (ini + fin) // 2  
        if v < lista[mitad]:  
            resultado = buscar(lista, v, ini, mitad)  
        else:  
            resultado = buscar(lista, v, mitad, fin)  
    return resultado
```

Por índices, como todo el curso

El arreglo completo se consulta con `buscar(lista, v, 0, len(lista))`. Cada llamado recibe de dónde a dónde trabaja y nadie copia nada: dividir cuesta $\Theta(1)$.

Traza

buscar(lista, 13, 0, 8) con $A = [1, 4, 6, 8, 10, 13, 20, 22]$

$[ini..fin)$	$mitad$	$A[mitad]$	Comparación	Decisión
$[0., 8)$	4	10	$13 \geq 10$	mitad derecha
$[4., 8)$	6	20	$13 < 20$	mitad izquierda
$[4., 6)$	5	13	$13 \geq 13$	mitad derecha
$[5., 6)$	—	—	$A[5] = 13$	True

De ocho posiciones a una en tres divisiones.

Ahora usted

Corra la traza con $v = 5$. ¿En qué intervalo de un elemento termina y qué responde?

¿Cuánto cuesta?

La recurrencia

Sobrevive una sola mitad y decidir cuál cuesta una comparación:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1), \quad T(1) = \Theta(1).$$

Expandiendo: cada nivel aporta una constante c , y los niveles se acaban cuando $n/2^k = 1$, o sea $k = \lg n$. El total es $c(\lg n + 1)$: $T(n) \in O(\lg n)$.

¿Cuánto cuesta?

Lineal contra binaria

n	Búsqueda lineal	Búsqueda binaria
10^3	1 000	10
10^6	1 000 000	20
10^9	1 000 000 000	30

Es el juego de adivinar el número, jugado por un algoritmo. CLRS la propone en el Ejercicio 2.3-5 (p. 39), con esta misma cota.

Teorema

Si $\text{lista}[\text{ini}..\text{fin})$ está ordenado y $\text{fin} - \text{ini} \geq 1$, entonces $\text{buscar}(\text{lista}, v, \text{ini}, \text{fin})$ produce el valor de $\exists p \in [\text{ini}..\text{fin}). \text{lista}[p] = v$.

Demostración: el caso base

Se procede por inducción estructural sobre el tamaño del intervalo, $n = \text{fin} - \text{ini}$.

Caso base ($n = 1$). El algoritmo devuelve $\text{lista}[\text{ini}] = v$, y la única posición del intervalo es ini : la salida coincide con la especificación. ✓

Correctitud

Demostración: el caso inductivo

Caso inductivo ($n > 1$). Primero, la partición separa y reduce: de $fin - ini \geq 2$ sale $ini < mitad < fin$, así que $[ini..mitad)$ y $[mitad..fin)$ son no vacíos y de tamaño menor que n . Por hipótesis de inducción, el llamado recursivo responde correctamente por su intervalo.

Falta ver que esa respuesta es la del intervalo completo, y ese es justo el argumento del descarte: si $v < A[mitad]$, la mitad derecha no puede tener testigos; si $v \geq A[mitad]$, todo testigo de la izquierda obliga a que $mitad$ mismo sea testigo derecho. En ambos casos, la mitad que sobrevive tiene la misma respuesta que el todo. ✓

Conclusión

Por inducción estructural, la invocación es correcta para todo intervalo; en particular $\varphi(0, N)$, que es la salida pedida. ■

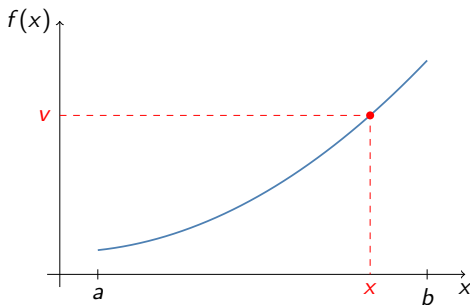
Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones**
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

El mismo problema, sin arreglo

Especificación

Entrada: una función f monótona creciente en un intervalo $[a, b]$ y un valor v con $f(a) \leq v \leq f(b)$. **Salida:** $x \in [a, b]$ tal que $f(x) = v$.

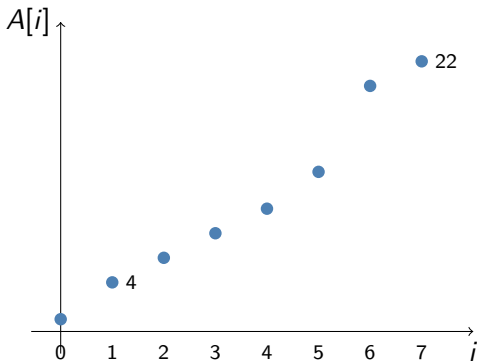


El mismo problema, sin arreglo

La misma jugada

Se evalúa f en la mitad del intervalo. Si $f(\text{mid}) < v$, la monotonía garantiza que x está a la derecha; si no, a la izquierda. Una mitad se descarta con certeza, igual que en el arreglo.

El puente entre los dos mundos



El puente entre los dos mundos

Un arreglo ordenado es una función monótona

Graficar $A = [1, 4, 6, 8, 10, 13, 20, 22]$ contra sus posiciones muestra una función $f : \{0, \dots, 7\} \rightarrow \mathbb{Z}$ con $f(i) = A[i]$, creciente. La búsqueda binaria es la bisección sobre un dominio discreto, y la bisección es la búsqueda binaria sobre un dominio continuo. Un solo algoritmo, dos presentaciones.

El dominio continuo pide tolerancia

Ya no hay igualdad exacta

Sobre los reales no se puede esperar $f(\text{mid}) = v$ al pie de la letra: los float redondean y el intervalo nunca queda de “un elemento”. La salida pasa a ser una **aproximación**: un x cuyo intervalo de incertidumbre mide menos que una tolerancia ε fijada de antemano.

```
def f(x):  
    # La funcion monotona del problema  
    return x * x * x + x  
  
def biseccion(v, a, b, eps):  
    # Aproxima x en [a, b] con f(x) = v, para f creciente  
    while b - a > eps:  
        mitad = (a + b) / 2  
        if f(mitad) < v:  
            a = mitad  
        else:  
            b = mitad  
    return (a + b) / 2
```

Ejemplo trabajado: $x^3 + x = 10$

biseccion(10, 0, 3, 1e-6)

$f(x) = x^3 + x$ es creciente, $f(0) = 0 \leq 10 \leq f(3) = 30$. Las primeras vueltas:

$[a, b]$	$mitad$	$f(mitad)$	Decisión
$[0, 3]$	1,5	4,875	< 10 : sube a
$[1,5, 3]$	2,25	13,64	≥ 10 : baja b
$[1,5, 2,25]$	1,875	8,47	< 10 : sube a
$[1,875, 2,25]$	2,0625	10,83	≥ 10 : baja b

El intervalo se cierra alrededor de $x = 2$, donde $2^3 + 2 = 10$.

Ejemplo trabajado: $x^3 + x = 10$

Cuántas vueltas

Cada vuelta parte el intervalo en dos, así que tras k vueltas mide $(b - a)/2^k$. Se necesita $(b - a)/2^k \leq \varepsilon$, es decir

$$k = \left\lceil \log_2 \frac{b - a}{\varepsilon} \right\rceil \quad \text{aquí: } \left\lceil \log_2 \frac{3}{10^{-6}} \right\rceil = 22 \text{ vueltas.}$$

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta**
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias

El problema de la leche

Especificación

Entrada: un arreglo $A[0..N)$ de enteros positivos —los envases de leche y su contenido— y un número $M \geq 1$ de contenedores disponibles. Los envases se vierten *en orden*, uno tras otro. **Salida:** la capacidad mínima de contenedor con la que basta con M contenedores.

Con $A = [5, 2, 2, 3, 2]$ y $M = 3$

Si los contenedores fueran de capacidad 9: el primero recibe $5 + 2 + 2 = 9$, el segundo $3 + 2 = 5$, y sobra uno. ¿Se puede con contenedores más pequeños? ¿Hasta dónde?

Pregunta

Aquí no hay arreglo ordenado ni función dada. ¿Dónde está lo monótono?

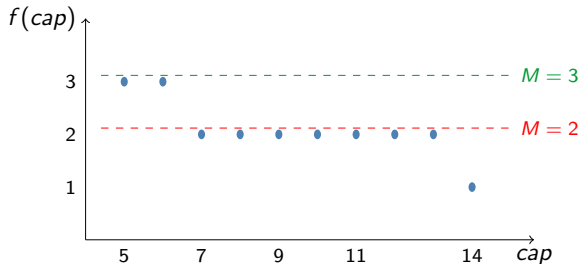
La función objetivo del problema

Nombrarla es la mitad del trabajo

$f(cap)$: contenedores que se necesitan si cada uno tiene capacidad cap .

Calcularla es directo: se vierte envase por envase y, cuando el contenedor actual no da más, se abre otro. Y es **decreciente**: más capacidad nunca exige más contenedores.

La función objetivo del problema



Reformulación de la salida

Se busca el menor cap con $f(\text{cap}) \leq M$. El rango donde vive la respuesta: de $\text{máx}(A)$ —el envase más grande tiene que caber— a $\text{sum}(A)$ —con todo en un contenedor sobra capacidad—. Para el ejemplo, $[5., 14]$.

De optimización a decisión

La transformación

La pregunta difícil “¿cuál es la capacidad óptima?” se cambia por la pregunta fácil “¿alcanza con capacidad cap ?”. La segunda se responde con un recorrido lineal, y como f es monótona, las respuestas forman dos bloques: primero los cap que no alcanzan, luego los que sí. El borde entre bloques es el óptimo, y un borde entre dos bloques ordenados se encuentra con búsqueda binaria.

El patrón, en general

Siempre que un problema de optimización admita una pregunta de decisión monótona en el parámetro que se optimiza, la respuesta se puede buscar binariamente sobre ese parámetro. En los juegos en línea el patrón aparece por todas partes.

La implementación

```
def contenedores(envases, cap):  
    # Contenedores de capacidad cap que exigen los envases, en orden  
    cuenta = 1  
    acumulado = 0  
    i = 0  
    while i < len(envases):  
        if acumulado + envases[i] <= cap:  
            acumulado = acumulado + envases[i]  
        else:  
            cuenta = cuenta + 1  
            acumulado = envases[i]  
        i = i + 1  
    return cuenta  
  
def capacidad_minima(envases, m):  
    # Menor capacidad de contenedor con la que bastan m contenedores  
    a = max(envases)  
    b = sum(envases)  
    while a < b:  
        mitad = (a + b) // 2  
        if contenedores(envases, mitad) <= m:  
            b = mitad
```

La implementación

```
    else:  
        a = mitad + 1  
    return a
```

La implementación, leída con calma

El chequeo

`contenedores` es la función objetivo hecha código: un recorrido $\Theta(n)$ que abre contenedor nuevo cuando el actual no da más.

La búsqueda

`capacidad_minima` busca el borde: si con *mitad* alcanza, la respuesta es *mitad* o algo menor (baja *b*); si no alcanza, es estrictamente mayor (sube *a* hasta *mitad* + 1). Al cerrarse el intervalo, *a* es el menor valor que alcanza.

El costo

Cada chequeo cuesta $\Theta(n)$ y hay $O(\lg(\text{sum}(A)))$ chequeos: $O(n \lg(\text{sum}(A)))$ en total. La búsqueda binaria corre sobre el rango de *respuestas*, no sobre el arreglo.

Traza

capacidad_minima([5, 2, 2, 3, 2], 3)

$[a..b]$	$mitad$	$f(mitad)$	$i \leq 3?$	Decisión
[5.,14]	9	2	sí	$b = 9$
[5.,9]	7	2	sí	$b = 7$
[5.,7]	6	3	sí	$b = 6$
[5.,6]	5	3	sí	$b = 5$

El intervalo se cierra en $a = b = 5$: con capacidad 5 alcanza, y menos no se puede porque el envase de 5 tiene que caber.

Ahora usted

Repita la traza con $M = 2$. La primera diferencia aparece en $mitad = 6$: ¿qué decide el algoritmo ahí y en qué respuesta termina?

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes**
- 6 Ejercicios
- 7 Referencias

Errores comunes

Errores comunes

Dónde se caen estos algoritmos

- **Un intervalo que no progresa:** con división entera, actualizar con $a = mitad$ en vez de $a = mitad + 1$ puede dejar el intervalo idéntico y el ciclo no termina. La terminación exige que *cada* rama reduzca.
- **Descartar sin monotonía:** sobre un arreglo sin ordenar o una función que sube y baja, el descarte puede botar la única solución. La monotonía no se asume: se argumenta, como con $f(cap)$.
- **Un rango inicial que no contiene la respuesta:** si la respuesta puede ser $\text{máx}(A)$ y el rango arranca en $\text{máx}(A) + 1$, ninguna búsqueda la encuentra. Las cotas del rango se justifican, no se adivinan.
- **Tolerancia mal escogida:** un ε más fino que la precisión del float deja el ciclo dando vueltas sin que $b - a$ baje. La tolerancia se fija según lo que pide el problema.

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios**
- 7 Referencias

Ejercicios

Ejercicios

Para practicar

- 1 Escriba la recurrencia de la búsqueda binaria y justifique la cota $\Theta(\lg n)$ del peor caso (CLRS, Ejercicio 2.3-5, p. 39).
- 2 Modifique buscar para que devuelva la *posición* de v , o -1 si no está. ¿Qué cambia en la función objetivo y en el caso base?
- 3 Para el ciclo de bisección: proponga el invariante que garantiza que la respuesta nunca se sale de $[a, b]$ y úselo para argumentar la correctitud.
- 4 *Copying Books* (UVa 714): K escribas copian libros contiguos; minimice las páginas del escriba más cargado. Plantee $f(t)$, su monotonía y el rango, y aplique el patrón de la leche.
- 5 *Athletics Track* (UVa 11646): halle el largo l de una pista con proporción dada cuyo perímetro sea 400. La función $f(l)$ es creciente: bisección continua.

Contenido

- 1 Repaso: divide y vencerás
- 2 Buscar en un arreglo ordenado
- 3 Bisección: de arreglos a funciones
- 4 Búsqueda sobre la respuesta
- 5 Errores comunes
- 6 Ejercicios
- 7 Referencias**

Referencias

-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, tercera edición, MIT Press, 2009. Sección 2.3 y Ejercicio 2.3-5, p. 39.
-  S. Halim, F. Halim y S. Effendy. *Competitive Programming 4*, 2018. Capítulo 3: búsqueda binaria sobre la respuesta.
-  C. Rocha. *Diseño y Análisis de Algoritmos*.